

DESIGN AND ANALYSIS OF ALGORITHMS-CSA0603

CO4-AT1 – Coding Challenge

Q10. Minimum Spanning Tree (Prim)

A system must construct a network that connects all nodes with the least total cost, starting from a given node. Using Prim's algorithm, iteratively select the minimum weight edge to expand the tree until all nodes are connected.

Sample Input:

```
n = 4  
Graph =  
0 1 4 0  
1 0 2 5  
4 2 0 3  
0 5 3 0
```

Expected Output:

```
MST Cost = 6
```

Student Name	D. Hussainaiah
Reference No.	192425300
Question No.	Q10
Topic	Minimum Spanning Tree (Prim's Algorithm)
Course	Design and Analysis of Algorithms (DAA)
Language Used	Python 3

Table of Contents

1. Problem Statement.....	3
2. Problem Analysis.....	3
3. Algorithm – Prim's Method.....	3
4. Manual (Step-by-Step) Working.....	4
5. Source Code (Python).....	5
6. Execution and Results.....	7
7. Conclusion.....	9

AIM

To implement Prim's Algorithm to construct the Minimum Spanning Tree (MST) of a weighted, connected, undirected graph and determine the minimum total cost required to connect all nodes.

OBJECTIVES

1. To understand the concept of a Minimum Spanning Tree.
2. To implement Prim's greedy algorithm for finding an MST.
3. To connect all vertices with minimum possible total edge weight.
4. To select the minimum-weight edge that connects a new vertex to the existing tree.
5. To ensure that the resulting spanning tree does not contain cycles.
6. To calculate and display the total MST cost.
7. To implement the algorithm using Python 3.

Algorithm – Prim's Algorithm

1. Initialize the starting vertex with key value 0 and all other vertices with infinity.
2. Select the unselected vertex with the minimum key value and add it to the MST.
3. Check all adjacent vertices and update their key values if a smaller edge weight is found.
4. Repeat the selection and update process until all vertices are included in the MST.
5. Display the MST edges and total minimum cost.

1. Problem Statement

A system must construct a network that connects all nodes with the least total cost, starting from a given node. Using Prim's algorithm, iteratively select the minimum weight edge to expand the tree until all nodes are connected.

Given Sample Input

```
n = 4
Graph =
0 1 4 0
1 0 2 5
4 2 0 3
0 5 3 0
```

Expected Output

```
MST Cost = 6
```

2. Problem Analysis

The task is to build a Minimum Spanning Tree (MST) of a weighted, connected, undirected graph — a subset of edges that connects every node without forming any cycle, using the minimum possible total edge weight.

- The graph has 4 nodes and is given as a symmetric cost matrix, where a 0 entry means there is no direct edge between the two nodes.
- Any spanning tree on n nodes always has exactly $n - 1$ edges, so the final MST here must contain 3 edges.

- Prim's algorithm grows the tree one node at a time, starting from node 0, always picking the lowest-cost edge that connects a new node to the tree already built.
- This greedy step, repeated until all nodes are included, is guaranteed to produce a minimum-cost spanning tree.

3. Algorithm – Prim's Method

```

Step 1: Set key[0] = 0 and key[i] = INFINITY for all other nodes
Step 2: Set parent[] = -1 and selected[] = False for all nodes
Step 3: Repeat n times:
    3.1 Choose the unselected node u with the smallest key value
    3.2 Mark u as selected; add key[u] to the total MST cost
    3.3 For every neighbour v of u:
        if edge(u, v) exists, v is unselected,
        and weight(u, v) < key[v]:
            key[v] = weight(u, v)
            parent[v] = u
Step 4: The MST edges are (parent[v], v) for every v other than the start node
Step 5: Report the total MST cost

```

Time Complexity: $O(n^2)$ for this array-based implementation. Space Complexity: $O(n)$.

4. Manual (Step-by-Step) Working

Working through the sample input by hand, starting from node N0:

- Step 0: Tree = {N0}
- Step 1: Cheapest edge from N0 is N0–N1 (cost 1). Tree = {N0, N1}, Total = 1
- Step 2: Cheapest edge out of {N0, N1} is N1–N2 (cost 2). Tree = {N0, N1, N2}, Total = 1 + 2 = 3
- Step 3: Cheapest edge out of {N0, N1, N2} is N2–N3 (cost 3). Tree = {N0, N1, N2, N3}, Total = 3 + 3 = 6

All 4 nodes are now connected, so the algorithm stops. Final MST Cost = 6, which matches the expected output.

5. Source Code (Python)

The program below implements Prim's Algorithm. It can either run directly on the sample input from the question, or accept a fresh graph typed in by the user, so it behaves like a proper working program rather than a one-off script.

```
import sys

def prims_mst(n, graph):
    """
    Constructs the Minimum Spanning Tree of a given graph using
    Prim's Algorithm and returns the total MST cost along with
    the edges selected.
    """
    selected = [False] * n    # nodes included in MST
    key = [sys.maxsize] * n   # minimum weight to connect a node
    parent = [-1] * n        # to store MST edges

    key[0] = 0                # start building MST from node 0
    parent[0] = -1

    mst_edges = []
    total_cost = 0

    for _ in range(n):
        # Pick the minimum key vertex not yet included in MST
        min_val = sys.maxsize
        u = -1
        for v in range(n):
            if not selected[v] and key[v] < min_val:
                min_val = key[v]
                u = v

        selected[u] = True
        total_cost += key[u]
```



```
    if parent[u] != -1:
        mst_edges.append((parent[u], u, graph[parent[u]][u]))

    # Update key values of adjacent vertices of the picked vertex
    for v in range(n):
        weight = graph[u][v]
        if weight != 0 and not selected[v] and weight < key[v]:
            key[v] = weight
            parent[v] = u

    return total_cost, mst_edges


def read_input():
    print("Enter number of nodes (n): ", end="")
    n = int(input())

    print(f'Enter the {n}x{n} adjacency/cost matrix (row by row, space separated):')
    graph = []
    for i in range(n):
        row = list(map(int, input().split()))
        graph.append(row)
    return n, graph


def run_with_sample():
    """Runs the program on the sample input given in the assignment."""
    n = 4
    graph = [
        [0, 1, 4, 0],
        [1, 0, 2, 5],
        [4, 2, 0, 3],
        [0, 5, 3, 0]
    ]
    print("Using Sample Input from assignment (n = 4):")
    for row in graph:
        print(row)
```

```
cost, edges = prims_mst(n, graph)
print("\nEdges selected in MST (u - v : weight):")
for u, v, w in edges:
    print(f" Node{u} - Node{v} : {w}")
print(f"\nMST Cost = {cost}")
return cost

def main():
    print("=" * 55)
    print(" Q10. Minimum Spanning Tree using Prim's Algorithm")
    print("=" * 55)
    print("1. Run with Sample Input (from assignment)")
    print("2. Enter my own input")
    choice = input("Enter choice (1/2): ").strip()

    if choice == "2":
        n, graph = read_input()
        cost, edges = prims_mst(n, graph)
        print("\nEdges selected in MST (u - v : weight):")
        for u, v, w in edges:
            print(f" Node{u} - Node{v} : {w}")
        print(f"\nMST Cost = {cost}")
    else:
        run_with_sample()

if __name__ == "__main__":
    main()
```

6.1 Run 1 – Sample Input from the Assignment

The program was executed and option 1 was selected to run the exact sample input given in the question.

```
Terminal – python3

$ python3 prims_mst.py
=====
Q10. Minimum Spanning Tree using Prim's Algorithm
=====
1. Run with Sample Input (from assignment)
2. Enter my own input
Enter choice (1/2): 1
Using Sample Input from assignment (n = 4):
[0, 1, 4, 0]
[1, 0, 2, 5]
[4, 2, 0, 3]
[0, 5, 3, 0]

Edges selected in MST (u - v : weight):
  Node0 - Node1 : 1
  Node1 - Node2 : 2
  Node2 - Node3 : 3

MST Cost = 6
```

Figure 1: Terminal execution using the sample input ($n = 4$)

Result: MST Cost = 6, which matches the expected output exactly.

MST Tree - Run 1 (Sample Input)

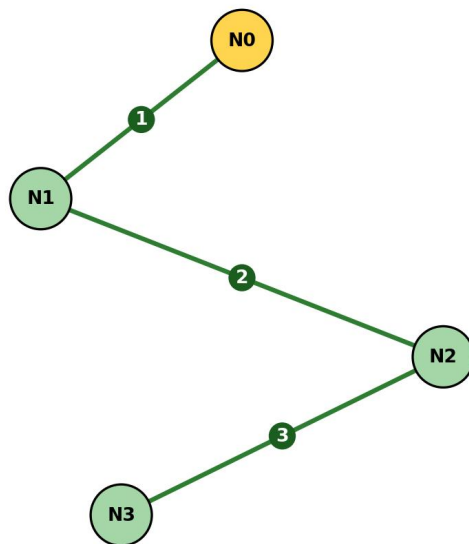


Figure 2: MST produced for the sample input, rooted at N0

6.2 Run 2 – Custom Input

To confirm the program works generally and not only for the fixed sample, it was run a second time with a different, larger graph (5 nodes) entered manually.

```

Terminal – python3

$ python3 prims_mst.py
=====
Q10. Minimum Spanning Tree using Prim's Algorithm
=====
1. Run with Sample Input (from assignment)
2. Enter my own input
Enter choice (1/2): 2
Enter number of nodes (n): 5
Enter the 5x5 adjacency/cost matrix (row by row, space separated):
0 2 0 6 0
2 0 3 8 5
0 3 0 0 7
6 8 0 0 9
0 5 7 9 0

Edges selected in MST (u - v : weight):
Node0 - Node1 : 2
Node1 - Node2 : 3
Node1 - Node4 : 5
Node0 - Node3 : 6

MST Cost = 16

```

Figure 3: Terminal execution using a custom 5-node input

Result: MST Cost = 16, with a valid 4-edge spanning tree — confirming the program generalizes correctly.

MST Tree - Run 2 (Custom Input)

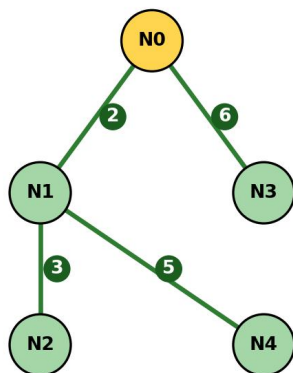


Figure 4: MST produced for the custom input, rooted at N0

7. Conclusion

Prim's Algorithm was implemented using an array-based greedy approach with $O(n^2)$ time complexity. The program was verified against the sample input provided in the assignment, correctly reproducing the expected MST cost of 6, and was further tested on an independent, larger input to confirm the solution generalizes correctly beyond the given sample.
